

Dynamic Programming

Model-Based Planning — Lecture 2/4

Etienne Chassaing

CentraleSupélec

Quiz time!

1. Dynamic Programming
2. Policy Evaluation
3. Policy Iteration
4. Value Iteration
5. Asynchronous Dynamic Programming
6. Convergence and Contraction

Lecture 1 : MDPs & Bellman

- Formalized MDP as (S, A, p, r, γ)
- Value functions: $v_\pi(s), q_\pi(s, a)$
- Optimal equations: $v_*(s), q_*(s, a)$
- Bellman equations on value functions

Today's key insight: instead of solving the Bellman equations directly (a linear system for v^π , and not even linear for v_* because of the max), we can solve them iteratively by repeatedly applying Bellman operations until convergence.

Why this matters: Later on, we'll be studying model-free control, and re-use this concept of iterative solution computation.

Lecture 2 : Dynamic Programming and Model-Based Planning

- We know the model $p(s'|s, a)$
- We know $r(s, a)$
- **Goal:** Compute v_* or π_* exactly

1. Dynamic Programming
2. Policy Evaluation
3. Policy Iteration
4. Value Iteration
5. Asynchronous Dynamic Programming
6. Convergence and Contraction

What Is Dynamic Programming?

- **Dynamic:** the problem has a sequential or temporal component.
- **Method:** solve a complex problem by breaking it down into simpler subproblems, **storing** their solutions, and **reusing** them instead of recomputing them.

Source: codingame.com/learn/dynamic-programming

What Is Dynamic Programming?

- **Dynamic:** the problem has a sequential or temporal component.
- **Method:** solve a complex problem by breaking it down into simpler subproblems, **storing** their solutions, and **reusing** them instead of recomputing them.

Source: codingame.com/learn/dynamic-programming

DP applies to problems with two properties:

Optimal substructure

- The principle of optimality applies
- An optimal solution decomposes into subproblems

Overlapping subproblems

- Subproblems recur many times
- Solutions can be cached and reused

MDPs satisfy both: the Bellman equation gives the recursive decomposition, and the value function stores and reuses the solutions.

Theorem: Principle of Optimality

A policy $\pi(a|s)$ achieves the optimal value from state s , $v^\pi(s) = v_*(s)$, if and only if:

- for any state s' reachable from s ,
- π achieves the optimal value from state s' : $v^\pi(s') = v_*(s')$.

Theorem: Principle of Optimality

A policy $\pi(a|s)$ achieves the optimal value from state s , $v^\pi(s) = v_*(s)$, if and only if:

- for any state s' reachable from s ,
- π achieves the optimal value from state s' : $v^\pi(s') = v_*(s')$.

Reading: an optimal policy is optimal *everywhere it can lead*: once we have reached s' , whatever happened before, the best we can do is to behave optimally from s' .

Consequence: the problem splits into a first step and a smaller problem starting from the successor state, which is the Bellman optimality equation:

$$v_*(s) = \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma v_*(s')]$$

Not to confuse with the policy improvement theorem (later): that one *compares* two policies, this one *characterizes* optimal ones.

Planning by Dynamic Programming

- DP assumes **full knowledge of the MDP** $\langle S, A, p, r, \gamma \rangle$.
- It is used for **planning** in an MDP.

Prediction

- **Input:** the MDP and a policy π
- **Output:** the value function v_π

Control

- **Input:** the MDP
- **Output:** the optimal value function v_* and an optimal policy π_*

1. Dynamic Programming
- 2. Policy Evaluation**
3. Policy Iteration
4. Value Iteration
5. Asynchronous Dynamic Programming
6. Convergence and Contraction

- A **policy** π says to you how to act : It is a mapping from states to actions .
- The **value function** $v^\pi(s)$ says to you how good being a state is for a given a policy: It gives the expected return when starting in state s and following policy π thereafter.

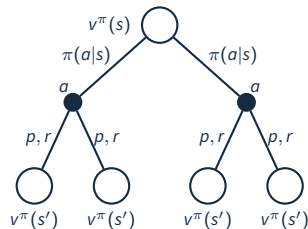
Policy Evaluation - Recall

- A **policy** π says to you how to act : It is a mapping from states to actions .
- The **value function** $v^\pi(s)$ says to you how good being a state is for a given a policy: It gives the expected return when starting in state s and following policy π thereafter.

⚠ : The value function **depends on the policy** π .

- **Bellman equation** is a way to link the value of a state to the values of its successor states. It is a recursive equation :

$$v_\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]$$



Backup diagram for v^π

In contrast to control, policy evaluation is a **prediction problem**: we just seek to evaluate a given policy.

Policy Evaluation - Principle

Goal: Given a policy π , compute $v_\pi(s)$ for all states s .

With known dynamics, Bellman equations give us $|S|$ **equations in** $|S|$ **unknowns:** the value functions $v^\pi(s_1), v^\pi(s_2), \dots, v^\pi(s_{|S|})$.

In principle we could solve this system directly, but for large state spaces this is **computationally expensive** ($O(|S|^3)$).

Policy Evaluation - Principle

Goal: Given a policy π , compute $v_\pi(s)$ for all states s .

With known dynamics, Bellman equations give us $|S|$ **equations in** $|S|$ **unknowns:** the value functions $v^\pi(s_1), v^\pi(s_2), \dots, v^\pi(s_{|S|})$.

In principle we could solve this system directly, but for large state spaces this is **computationally expensive** ($O(|S|^3)$).

Better approach: Use **iterative methods**. Repeatedly apply the Bellman expectation operator until convergence.

$$v_\pi(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]$$

Key idea:

Repeatedly apply the Bellman expectation operator. Each iteration refines the value estimates, and the sequence converges to v^π .

Policy Evaluation — Pseudocode

Algorithm for iterative policy evaluation is the following:

Policy Evaluation

Input: Policy π , threshold $\epsilon > 0$

Initialize $v(s) = 0$ for all $s \in S$

repeat

$\Delta \leftarrow 0$

for each $s \in S$:

$v_{\text{old}}(s) \leftarrow v(s)$

$v(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v(s')]$

$\Delta \leftarrow \max(\Delta, |v(s) - v_{\text{old}}(s)|)$

until $\Delta < \epsilon$

Output: Value function v^π

Cost of one sweep: $O(|S|^2|A|)$

every s , every a , every s' (one expected reward $\bar{r}(s, a, s')$ per transition, precomputed).

In place: one array v , so later states in a sweep already use this sweep's new values.

Policy Evaluation — Gridworld Example from Sutton & Barto

Setup: A simple gridworld with 14 non-terminal states $S = \{1, 2, \dots, 14\}$ arranged in a grid.

- **Actions:** $A = \{\text{up, down, right, left}\}$, deterministic transitions, except at boundaries (action unchanged)
- **Reward:** $r(s, a, s') = -1$ for all transitions (cost per step)
- **Task:** Undiscounted, episodic (one terminal state)
- **Policy:** Equiprobable random: $\pi(a|s) = 0.25$ for all a, s

Result: Iterative policy evaluation converges to $v^\pi(s)$, what can we expect this value function to look like? Think about the problem type, the reward structure, the symmetry of the problem, the policy ...

See the convergence evolve on the next slide.



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

Policy Evaluation - Value Function Evolution

Equiprobable random policy, $\gamma = 1$, $r = -1$ per step. Values converge to v^π as $k \rightarrow \infty$.

$s = 1$ (orange state)

$$v_1(1) = \frac{1}{4} [1 \cdot (-1 + v_0(1)) + 1 \cdot (-1 + v_0(5)) + 1 \cdot (-1 + v_0(0)) + 1 \cdot (-1 + v_0(2))] = -1$$

$s = 6$ (cyan state)

$$v_2(6) = \frac{1}{4} [1 \cdot (-1 + v_1(2)) + 1 \cdot (-1 + v_1(10)) + 1 \cdot (-1 + v_1(5)) + 1 \cdot (-1 + v_1(7))] = -2.0$$

Question : When would you stop iterating to get enough conclusions?

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 0$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 1$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 2$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 3$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 10$

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

$v^\pi (k = \infty)$

Try it in the [Gridworld space](#): play the sweeps and click a state to see its Bellman backup.

1. Dynamic Programming
2. Policy Evaluation
- 3. Policy Iteration**
4. Value Iteration
5. Asynchronous Dynamic Programming
6. Convergence and Contraction

From Evaluation to Improvement

- **Key insight:** Once we know $v^\pi(s)$ for all states, we can measure the impact of taking a **different action** than the policy recommends.

From Evaluation to Improvement

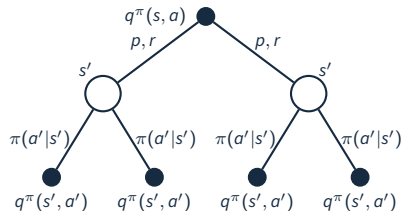
- **Key insight:** Once we know $v^\pi(s)$ for all states, we can measure the impact of taking a **different action** than the policy recommends.
- **Recall:** the action-value function under policy π is

$$\begin{aligned} q^\pi(s, a) &= \mathbb{E}[R_t \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) [r + \gamma v^\pi(s')] \end{aligned}$$

the **expected return** if we take action a in state s , then follow π forever after.

- **Relationship:**

$$v^\pi(s) = \sum_a \pi(a|s) q^\pi(s, a)$$



Backup diagram for q^π

From Evaluation to Improvement

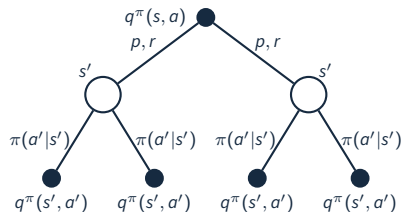
- **Key insight:** Once we know $v^\pi(s)$ for all states, we can measure the impact of taking a **different action** than the policy recommends.
- **Recall:** the action-value function under policy π is

$$\begin{aligned} q^\pi(s, a) &= \mathbb{E}[R_t \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) [r + \gamma v^\pi(s')] \end{aligned}$$

the **expected return** if we take action a in state s , then follow π forever after.

- **Relationship:**

$$v^\pi(s) = \sum_a \pi(a|s) q^\pi(s, a)$$



Backup diagram for q^π

Improvement question

If $\exists a \neq \pi(s)$ such that $q^\pi(s, a) > v^\pi(s)$, taking a is **better than following π** at state s .

This motivates the policy improvement step: greedify with respect to q^π .

Theorem: Policy Improvement Theorem

Let π and π' be two policies. If

$$q^\pi(s, \pi'(s)) \geq v^\pi(s) \quad \forall s \in S$$

then π' is **at least as good as** π : $v^{\pi'}(s) \geq v^\pi(s)$ for all s .

Proof: in the exercises (bonus) or in the appendix.

Theorem: Policy Improvement Theorem

Let π and π' be two policies. If

$$q^\pi(s, \pi'(s)) \geq v^\pi(s) \quad \forall s \in S$$

then π' is **at least as good as** π : $v^{\pi'}(s) \geq v^\pi(s)$ for all s .

Proof: in the exercises (bonus) or in the appendix.

Greedy policy improvement: We apply this by computing the greedy policy (w.r.t to the Bellman optimality equation) for each state:

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v^\pi(s')] \quad \forall s \in S$$

This is equivalent to $\pi'(s) = \arg \max_a q^\pi(s, a)$.

Theorem: Policy Improvement Theorem

Let π and π' be two policies. If

$$q^\pi(s, \pi'(s)) \geq v^\pi(s) \quad \forall s \in S$$

then π' is **at least as good as** π : $v^{\pi'}(s) \geq v^\pi(s)$ for all s .

Proof: in the exercises (bonus) or in the appendix.

Greedy policy improvement: We apply this by computing the greedy policy (w.r.t to the Bellman optimality equation) for each state:

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v^\pi(s')] \quad \forall s \in S$$

This is equivalent to $\pi'(s) = \arg \max_a q^\pi(s, a)$. **Convergence:** If $\pi' = \pi$ (policy is stable), then:

$$v^\pi(s) = \max_a q^\pi(s, a)$$

which means that π satisfied the Bellman **optimality** equation. Hence $v^\pi = v_*$ and $\pi = \pi_*$.

Policy Iteration

Key idea: Greedily improving with respect to v^π is guaranteed to make the policy at least as good, and continuing should reach the optimal policy.

Policy Iteration

Key idea: Greedily improving with respect to v^π is guaranteed to make the policy at least as good, and continuing should reach the optimal policy. **Policy Iteration = Full Evaluation + Greedy Improvement:**

1. **Policy Evaluation:** Compute v^π (iterate to convergence or near-convergence)
2. **Policy Improvement:** Greedify: $\pi'(s) = \arg \max_a q^\pi(s, a)$
3. Repeat outer loop until policy stabilizes: $\pi' = \pi$

Policy Iteration

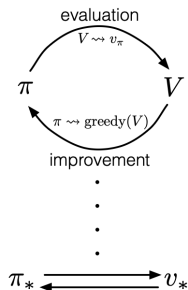
Key idea: Greedily improving with respect to v^π is guaranteed to make the policy at least as good, and continuing should reach the optimal policy. **Policy Iteration = Full Evaluation + Greedy Improvement:**

1. **Policy Evaluation:** Compute v^π (iterate to convergence or near-convergence)
2. **Policy Improvement:** Greedify: $\pi'(s) = \arg \max_a q^\pi(s, a)$
3. Repeat outer loop until policy stabilizes: $\pi' = \pi$

The iteration chain: (E: Evaluation, I: Improvement)

$$\pi_0 \xrightarrow{\text{E}} v^{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v^{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*, \pi_*$$

Because a finite MDP has a finite number of policies, this process is **guaranteed to converge** to the optimal value function (unique) and optimal policy (might be non-unique) in a **finite** number of iterations.



Policy Iteration

Initialize π arbitrarily (e.g., random)

repeat

Policy Evaluation:

Compute $v^\pi(s)$ for all s via iterative Bellman:

$v(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v(s')]$ until convergence

Policy Improvement:

$\pi_{\text{old}} \leftarrow \pi$

for each $s \in S$:

$\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v(s')]$

if $\pi = \pi_{\text{old}}$ **then break**

until policy is stable

Full policy evaluation (itself iterative) can be slow. Each outer loop guarantees improvement or convergence.

Policy Iteration - Evolution of the Policy (1/2)

Idea: greedify w.r.t. v_k at each k , without waiting for full convergence.

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

v_0

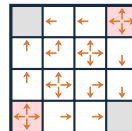
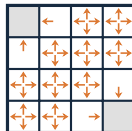
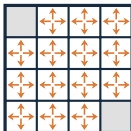
0.0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	0.0

v_1

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

v_2

greedy(v_k):



Policy Iteration — Evolution of the Policy (2/2)

Continuing further: the greedy policy stabilizes well before v_k itself converges.

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

v_3

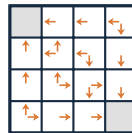
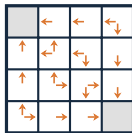
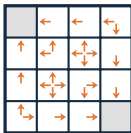
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

v_{10}

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

$v^{\pi_0} (k = \infty)$

greedy(v_k):



Policy Iteration — Evolution of the Policy (2/2)

Continuing further: the greedy policy stabilizes well before v_k itself converges.

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

v_3

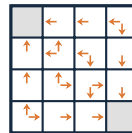
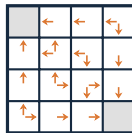
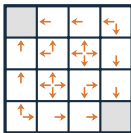
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

v_{10}

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

$v^{\pi_0} (k = \infty)$

greedy(v_k):



Result: by $k = 3$ the greedy policy already matches π_* ; from $k = 10$ onward it settles into one consistent policy and never changes again.

Try it in the [Gridworld space](#): step through the policy iteration.

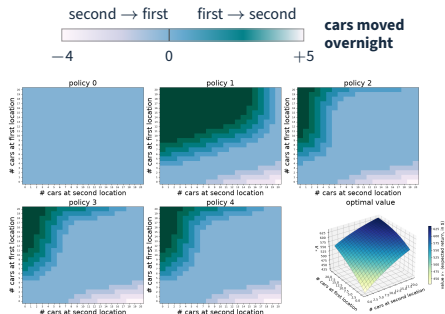
How to Picture a Policy: Jack's Car Rental

Sutton & Barto, Example 4.2: the textbook's worked example for policy iteration.

Setup: Jack manages two rental locations.

- **State:** number of cars at each location (up to 20 each)
- **Action:** move up to 5 cars between locations **overnight**
- **Reward:** \$10 per car rented, $-\$2$ per car moved
- **Dynamics:** requests and returns follow **Poisson distributions** (known rates), so $p(s'|s, a)$ is fully known $\Rightarrow$ this is a DP problem.

Poisson law of rate λ : number of events N in a fixed period (a day), independent events, on average λ :
 $\mathbb{P}(N = n) = \lambda^n e^{-\lambda} / n!$, $\mathbb{E}[N] = \text{Var}(N) = \lambda$.



Each cell is a state (cars at each location); its **colour is the action** (cars moved overnight, see the legend above). S&B Fig. 4.2.

What the book shows: from $\pi_0 =$ “never move cars”, policy iteration is run to convergence. The boundaries between “move n cars” regions shift and stabilize by π_4 .

1. Dynamic Programming
2. Policy Evaluation
3. Policy Iteration
- 4. Value Iteration**
5. Asynchronous Dynamic Programming
6. Convergence and Contraction

Value Iteration

Idea: Skip full policy evaluation. Instead, just apply the **Bellman optimality operator**:

$$v_{k+1}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

Value Iteration

Idea: Skip full policy evaluation. Instead, just apply the **Bellman optimality operator**:

$$v_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v_k(s')]$$

Value Iteration = Direct optimization without explicit policy:

1. Initialize $v(s) = 0$ for all s
2. **repeat** the value update:
 - ▶ For each state s , Until $\|v_{\text{new}} - v\| < \epsilon$ (convergence)

$$v_{\text{new}}(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$$

If needed, extract (pseudo) optimal policy :

$$\pi_*(s) \approx \arg \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v_*(s')]$$

Advantage: Single, simple update. **Tradeoff:** May need more iterations than policy iteration.

Value Iteration

Initialize $v(s) = 0$ for all $s \in S$

repeat

$\Delta \leftarrow 0$

for each $s \in S$:

$v_{\text{old}}(s) \leftarrow v(s)$

$v(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$

$\Delta \leftarrow \max(\Delta, |v(s) - v_{\text{old}}(s)|)$

until $\Delta < \epsilon$

return $\pi_*(s) = \arg \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$

Value Iteration

Initialize $v(s) = 0$ for all $s \in S$

repeat

$\Delta \leftarrow 0$

for each $s \in S$:

$v_{\text{old}}(s) \leftarrow v(s)$

$v(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$

$\Delta \leftarrow \max(\Delta, |v(s) - v_{\text{old}}(s)|)$

until $\Delta < \epsilon$

return $\pi_*(s) = \arg \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$

Cost of one sweep: $O(|S|^2|A|)$

(same assumption on the rewards)

Number of sweeps: the Bellman *optimality* operator is also a γ -contraction, so the error shrinks by a factor γ each sweep.

vs. policy iteration: one sweep per improvement instead of a full evaluation. Cheaper iterations, but usually more of them.

Output policy: with one extra sweep (the $\arg \max$) extracts π_* .

Value Iteration — Values Converging, Then the Policy

Bellman optimality update $v_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')]$, from $v_0 = 0$:

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

v_0

0.0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	0.0

v_1

0.0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-1
-2	-2	-1	0.0

v_2

0.0	-1	-2	-3
-1	-2	-3	-2
-2	-3	-2	-1
-3	-2	-1	0.0

v_3

0.0	-1	-2	-3
-1	-2	-3	-2
-2	-3	-2	-1
-3	-2	-1	0.0

$v_4 = v_3$

Value Iteration — Values Converging, Then the Policy

Bellman optimality update $v_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')]$, from $v_0 = 0$:

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

v_0

0.0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	0.0

v_1

0.0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-1
-2	-2	-1	0.0

v_2

0.0	-1	-2	-3
-1	-2	-3	-2
-2	-3	-2	-1
-3	-2	-1	0.0

v_3

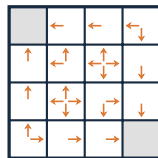
0.0	-1	-2	-3
-1	-2	-3	-2
-2	-3	-2	-1
-3	-2	-1	0.0

$v_4 = v_3$

Extract the policy with one greedy sweep

on $v_* = v_3$:

$$\pi_*(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')]$$



π_*

Same π_* as the one Policy Iteration converged to (previous slide): both algorithms reach the same v_* , hence the same optimal actions.

Try it in the [Gridworld space](#): step through the value iteration.

Policy Iteration vs Value Iteration

	Policy Iteration	Value Iteration
Convergence guarantee	Yes (π_* in finite steps)	Yes (v_* asymptotically)
Speed (typical)	Fewer outer iterations	More iterations but simpler update
Computation per step	One full PE sweep	One Bellman max
When to use	Medium-sized MDPs	Large state spaces

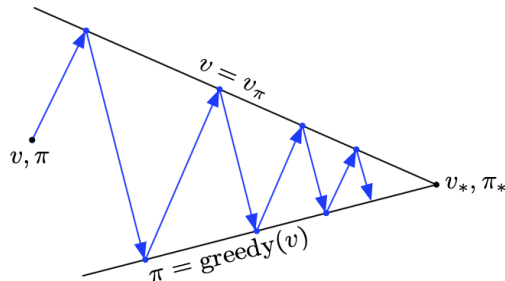
In practice: Most modern RL uses value iteration or variants (TD learning, etc.)

Generalized Policy Iteration

Core idea: Both Policy Iteration and Value Iteration are instances of the same framework. **Generalized Policy Iteration** is a generic name to describe the interaction of evaluation and improvement.

- **Policy Evaluation:** Estimate v^π **with any** policy evaluation method
- **Policy Improvement:** Greedify w.r.t. v^π to get a new policy π' **with any** policy improvement method

Value iteration can be seen as a special case of policy iteration where the evaluation step is truncated to a single update per state.



1. Dynamic Programming
2. Policy Evaluation
3. Policy Iteration
4. Value Iteration
- 5. Asynchronous Dynamic Programming**
6. Convergence and Contraction

The Problem: Sweeps Don't Scale

One limitation of the algorithms so far: Every iteration requires a **sweep** over the **entire** state set S .

If $|S|$ is very large, a single sweep can be prohibitively expensive.

Example: Rubik's Cube

- A $3 \times 3 \times 3$ cube has about 4.3×10^{19} states, and its model is fully known
- Even at 10^6 state updates per second...
- ...a single sweep would take **over a million years**

Question: Do we really need to update *every* state before making progress?

Synchronous value iteration

Two copies of the value function ($v_{\text{old}}, v_{\text{new}}$):

$$v_{\text{new}}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\text{old}}(s')]$$
$$v_{\text{old}} \leftarrow v_{\text{new}}$$

- Every update in a sweep reads the **same** v_{old}
- One sweep is exactly $v_{k+1} = \mathcal{T}^* v_k$
- The order of the states does not matter

Synchronous value iteration

Two copies of the value function ($v_{\text{old}}, v_{\text{new}}$):

$$v_{\text{new}}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\text{old}}(s')]$$
$$V_{\text{old}} \leftarrow V_{\text{new}}$$

- Every update in a sweep reads the **same** V_{old}
- One sweep is exactly $v_{k+1} = \mathcal{T}^* v_k$
- The order of the states does not matter

In-place value iteration

One copy of the value function v :

$$v(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v(s')]$$

- An update **overwrites** $v(s)$ immediately
- States updated later in the same sweep already see the **new** values
- The order of the states now matters

The value iteration pseudocode seen earlier is the in-place version.

Idea: In-place iterative DP algorithms that are **not** organized as systematic sweeps of the state set.

- States are updated **in any order**, using whatever values of other states happen to be available
- Some states may be updated several times before others are updated once
- **Great flexibility** in selecting which states to update, and when

One constraint remains: to converge correctly, the algorithm must keep updating **all** states; it cannot ignore any state forever after some point.

Asynchronous DP: Using the Flexibility

Avoiding sweeps does not mean less computation, but it lets us **spend** the computation where it helps:

- **Choose** which states to update, to improve the rate of progress
- **Order** the updates so that value information propagates efficiently from state to state
- Update some states **less often** than others, or **skip** those irrelevant to optimal behavior
- **Intermix** computation with real-time interaction: run DP while an agent is actually experiencing the MDP, and use its experience to **focus** the updates on the states it visits

But the model is still assumed known

Every update above still computes $\sum_{s',r} p(s',r|s,a) [r + \gamma v(s')]$: we need p and r . The agent's experience only decides *where* to update, not *what* the model is.

Next lecture: remove this assumption. Monte Carlo and TD methods learn from sampled transitions (s, a, r, s') , with no model.

1. Dynamic Programming
2. Policy Evaluation
3. Policy Iteration
4. Value Iteration
5. Asynchronous Dynamic Programming
- 6. Convergence and Contraction**

The Bellman Operator as a Contraction

Why do these algorithms converge?

Define the **Bellman optimality operator**:

$$(\mathcal{T}^*v)(s) = \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$$

Theorem (Contraction Mapping): For discount factor $\gamma < 1$, $\mathcal{T}^*$ is a γ -contraction in sup-norm:

$$\|\mathcal{T}^*v_1 - \mathcal{T}^*v_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty$$

Consequence (Banach Fixed-Point Theorem):

- $\mathcal{T}^*$ has a unique fixed point v_*
- Repeated application: $v_k = \mathcal{T}^*v_{k-1}$ converges to v_*
- Convergence rate: $\|v_k - v_*\| \leq \gamma^k \|v_0 - v_*\|$ (exponential)

Sources : Full proof: Bertsekas & Tsitsiklis, *Neuro-Dynamic Programming* (1996), or <https://cfml.se/bellman-operators-are-contractions/> for a self-contained write-up.

Where This Is Going: AlphaZero

Go also has a fully known model: the rules are exact, so in principle it *is* a planning problem, just like Jack's Car Rental.

The catch: the state space is about 10^{170} states. Sweep-based DP (and even async DP) is hopeless at this scale.

AlphaZero's answer:

- Replace exhaustive backups with **Monte Carlo Tree Search**, a form of *planning* that expands only the most promising states, guided by a value estimate (conceptually a Bellman backup done selectively, not exhaustively)
- Replace the exact value/policy tables with a **neural network** $(\hat{v}, \hat{\pi})$ trained by self-play

Same idea, different scale: Bellman-style backups + policy improvement, but **approximated** and **selective** instead of exact and exhaustive.

This is exactly the tension motivating the rest of the course: exact DP $\rightarrow$ sample-based, approximate methods (Monte Carlo, TD, function approximation).

Key Takeaways

- **Dynamic Programming** solves MDPs when the model $p(s'|s, a)$ is known
- **Policy Evaluation:** Solve for v^π given π
- **Policy Iteration:** Repeat evaluation + greedy improvement $\rightarrow$ converges to π_*
- **Value Iteration:** Apply Bellman max directly $\rightarrow$ simpler, converges to v_*
- **Convergence guarantee:** Bellman operator is a γ -contraction ($\gamma < 1$)
- **Next:** Model-free learning (no need to know p) via Monte Carlo, TD, Q-learning

Questions?

Your feedback on this lecture is welcome

etienne.chassaing@centralesupelec.fr



Survey form (anonymous feedback):

<https://forms.gle/L7ewxBUTUaZbpcya8>



Next: Kahoot quiz at start of Lecture 3!

About this course: I do use LLMs/AI to create this course, with the following uses:

- Format $\text{\LaTeX}$ slides and exercises
- Find relevant sources
- Format equations and notations

However, plan and ideas are human-based, and every content has been carefully reviewed. If you find some errors, or inconsistencies, please report them to me.

What about your own LLM usage?

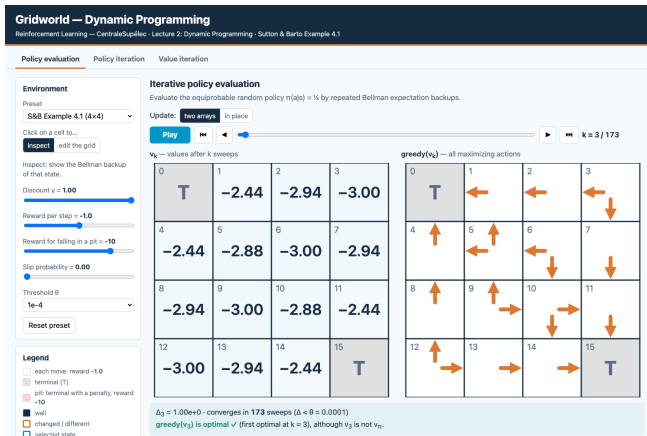
Of course you also have the right to use LLMs, but don't forget you're in a learning step. Learning process goes through effort and the newer the subject, the more one needs empty-sheet recall.

* See: Desirable difficulty

* Chi & Wylie (2014), ICAP hypothesis

7. Appendix

The Gridworld Space



Policy evaluation, policy iteration and value iteration on the 4×4 gridworld, one sweep at a time:

[Gridworld space.](#)

Policy Improvement Theorem - Proof

Idea: keep applying the hypothesis $q^\pi(s, \pi'(s)) \geq v^\pi(s)$ one step further each time, and unroll.

$$\begin{aligned}v^\pi(s) &\leq q^\pi(s, \pi'(s)) \\&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v^\pi(S_{t+1}) \mid S_t = s] \\&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q^\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}_{\pi'}[R_{t+2} + \gamma v^\pi(S_{t+2})] \mid S_t = s] \\&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v^\pi(S_{t+2}) \mid S_t = s] \\&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v^\pi(S_{t+3}) \mid S_t = s] \\&\vdots \\&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \mid S_t = s] \\&= v^{\pi'}(s)\end{aligned}$$

Each $\leq$ replaces v^π by $q^\pi(\cdot, \pi'(\cdot))$, which is at least as large by hypothesis.
Following π' for one more step at a time, we end up following π' forever.